

CodroidCPP SDK 手册

版本: 2.1.7 | 命名空间: `Codroid`

目录

#	章节	说明
1	快速上手	构建、连接并运行第一个程序
2	核心概念	生命周期、TCP 模型、单位约定、异常处理
3	CodroidClient API	CodroidClient 完整 API 参考
4	运动 API	JointPoint、CartesianPoint、MoveInstruction、MotionWaitOptions、枚举
5	数据类型与枚举	CommandResult、ClientRealtimeState、RobotFrame、异常类
6	CRI 实时数据与控制	CriRealtimeDispatcher、TrajectoryGenerator、TrajectoryRequest
7	IO 与寄存器	DI/DO/AI/AO 操作、寄存器读写
8	辅助工具	主题订阅、全局变量、运动学、ConsoleUtf8

环境要求

平台	编译器	构建工具
Linux	GCC 9+ / Clang 10+	CMake 3.14+
Windows	MSVC 2019/2022/2026	CMake 3.14+
Windows	MinGW-w64 (MSYS2)	CMake 3.14+

依赖项

- **Asio** (独立版, 非 Boost) — 网络通信
- **nlohmann/json** — JSON 序列化
- **GoogleTest** (可选) — 单元测试

构建

Linux

```
chmod +x build_linux.sh
./build_linux.sh
```

产物在 `build_linux/` (包含 `libCodroid.so` 与示例程序)。

Windows (MSVC)

```
build_msvc.bat
```

脚本支持：

- 1： Visual Studio 2019
- 2： Visual Studio 2022
- 3： Visual Studio 2026

产物在 build_msvc/Debug 与 build_msvc/Release。

Windows (MinGW)

```
build_mingw.bat
```

产物在 build_mingw/。

API 命名约定

所有公共方法使用 **PascalCase** 命名，与 C# 和 Python SDK 保持一致。

```
// 正确
robot.ConnectRemoteAndSwitchOn("192.168.8.136");
int di = robot.GetDi(0);
robot.MovJ(joints, 40, 100);
```

单位约定

层级	线性	角度
SDK 公共 API	mm	deg（度）
TCP JSON 协议	mm	deg
CRI UDP 二进制（线路层）	m	rad（弧度）
ClientRealtimeState（已解析）	mm	deg

CriRealtimeDispatcher 在 convert_to_si=true（默认）时会自动将 mm/deg 转换为 m/rad。

固件要求

本 SDK 所有对外接口均要求控制器固件 $\geq 2.3.3.43$ 。

代码常量见 Codroid::MinControllerFirmware（CodroidDefine.h）。

快速上手

构建 SDK

Linux

```
# 克隆仓库
git clone <repository-url>
cd CodroidCPP

# 构建
chmod +x build_linux.sh
./build_linux.sh
```

产物在 `build_linux/`：

- `libCodroid.so` — 动态库
- `examples/` — 示例程序

Windows (MSVC)

```
REM 克隆仓库后
cd CodroidCPP

REM 构建
build_msvc.bat
```

选择 Visual Studio 版本：

- `1`：Visual Studio 2019
- `2`：Visual Studio 2022
- `3`：Visual Studio 2026

产物在 `build_msvc/Debug` 和 `build_msvc/Release`。

Windows (MinGW)

```
cd CodroidCPP
build_mingw.bat
```

产物在 `build_mingw/`。

最小示例

连接控制器，读取数字输入，写入数字输出，然后断开。

```
#include "Codroid/client.hpp"
```

```

#include <iostream>

int main() {
    Codroid::CodroidClient robot;

    // 连接、切换远程、上电
    if (!robot.ConnectRemoteAndSwitchOn("192.168.8.136", 9001, "192.168.8.150")) {
        std::cerr << "连接失败" << std::endl;
        return 1;
    }

    // 读取 DI 端口 0
    int di0 = robot.GetDi(0);
    std::cout << "DI 0 = " << di0 << std::endl;

    // 将 DI 值写入 DO 端口 10
    robot.SetDo(10, di0);

    // 断开连接
    robot.Disconnect();
    return 0;
}

```

完整工作流示例

```

#include "Codroid/client.hpp"
#include "Codroid/cri_realtime_dispatcher.hpp"
#include "Codroid/trajectory_generator.hpp"
#include <iostream>

int main() {
    // Windows 控制台 UTF-8 支持
#ifdef _WIN32
    Codroid::ConsoleUtf8::InitConsoleUtf8();
#endif

    Codroid::CodroidClient robot;

    // 1. 连接
    if (!robot.ConnectRemoteAndSwitchOn("192.168.8.136", 9001, "192.168.8.150")) {
        std::cerr << "连接失败" << std::endl;
        return 1;
    }

    // 2. IO 操作
    int di0 = robot.GetDi(0);
    robot.SetDo(10, di0);

    // 3. 寄存器
    double regValue = robot.GetRegisterValue(49100);
    robot.SetRegisterValue(49100, regValue + 1);
}

```

```

// 4. 运动
auto joints = Codroid::JointPoint::Degrees({0, 0, 90, 0, 90, 0});
robot.MovJ(joints, 40, 100);

// 5. 阻塞运动
auto state = robot.GetRobotRealtimeState();
auto target = Codroid::CartesianPoint::MmDegWithRef(
    {400, 0, 300, 180, 0, 0},
    state.joint_position
);
robot.MovLSync(target, 150, 500);

// 6. 断开
robot.Disconnect();
return 0;
}

```

运行示例程序

```

# Linux
cd build_linux
./examples/01_basic_usage 192.168.8.136

# Windows (MSVC)
cd build_msvc\Release
01_basic_usage.exe 192.168.8.136

```

错误处理

TCP 指令失败时的行为取决于 `SetThrowOnCommandError` 设置：

默认模式（不抛异常）

```

Codroid::CodroidClient robot;
robot.Connect("192.168.8.136");

auto result = robot.SetDo(999, 1); // 无效端口
if (!result.Ok()) {
    std::cerr << "控制器错误: " << result.error_msg << std::endl;
}

```

抛异常模式

```
robot.SetThrowOnCommandError(true);

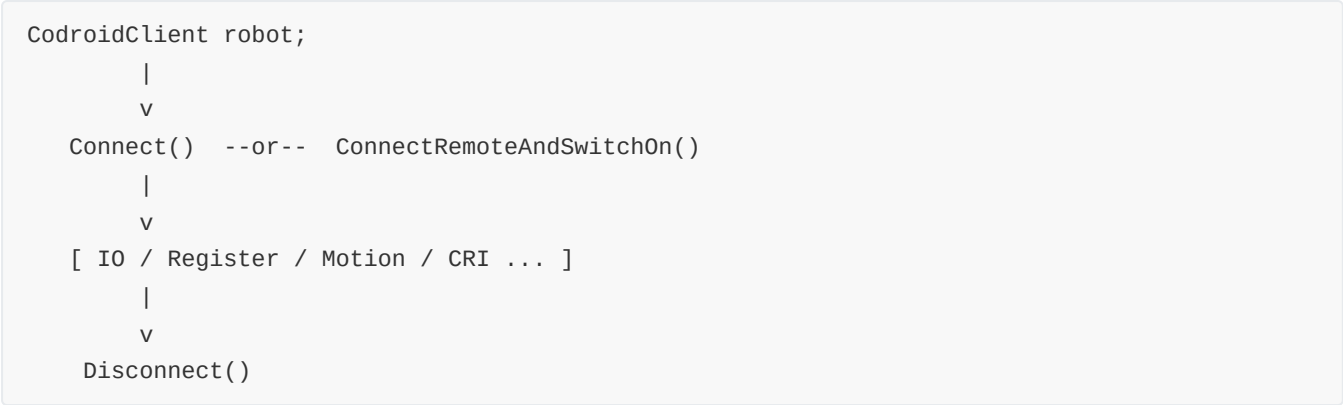
try {
    robot.SetDo(999, 1); // 无效端口
} catch (const Codroid::CodroidCommandException& ex) {
    std::cerr << "控制器错误: " << ex.controller_error() << std::endl;
}
```

异常类型

异常	条件
CodroidCommandException	控制器返回 err 字段
CodroidException	通用运行时错误
std::runtime_error	标准库异常

核心概念

CodroidClient 生命周期



```
Codroid::CodroidClient robot;

try {
    robot.ConnectRemoteAndSwitchOn("192.168.8.136", 9001, "192.168.8.150");
    // ... 使用 robot ...
} catch (...) {
    robot.Disconnect(); // 始终调用
    throw;
}
robot.Disconnect();
```

构造函数

```
Codroid::CodroidClient robot;
```

- 默认构造，不连接
- 调用 `Connect` 或 `ConnectRemoteAndSwitchOn` 建立连接
- TCP 端口默认 **9001**

属性

方法	返回类型	说明
<code>GetRobotRealtimeState()</code>	<code>ClientRealtimeState</code>	CRI 数据快照的线程安全副本
<code>GetCriUdpListenPort()</code>	<code>int</code>	CRI UDP 监听端口

回调

```
robot.SetCriDataReceived([](const Codroid::ClientRealtimeState& data) {
    std::cout << "关节: ";
    for (auto j : data.joint_position) std::cout << j << " ";
    std::cout << std::endl;
});
```

每当解析完一个有效的 CRI UDP 帧后触发。回调在内部接收线程上执行，避免长时间阻塞。

TCP 指令模型

每个与控制器通信的 SDK 方法都遵循以下模式：

- 1. SDK 分配唯一的 `id`
- 2. SDK 将 `{ id, ty, db }` 序列化为 JSON 并通过 TCP 发送
- 3. 控制器响应 `{ id, ty, db, err }`
- 4. SDK 通过 `id` 匹配响应
- 5. 若 `err` 非空则 `CommandResult::error_msg` 非空（或抛异常）
- 6. 若 10 秒内未收到响应则超时

CommandResult

```
struct CommandResult {
    int id = 0;           // 请求 id
    std::string ty;       // 响应类型
    std::string error_msg; // 错误信息（空表示成功）
    std::string raw_json;  // 完整响应 JSON

    bool ok() const noexcept; // error_msg 为空返回 true
};
```

大多数方法返回 `CommandResult`。检查 `ok()` 判断是否成功。

单位约定

SDK 公共 API 使用毫米和度。这与 TCP JSON 协议一致。

上下文	线性	角度
SDK API、TCP JSON	mm	deg
CRI UDP 线路格式	m	rad
<code>ClientRealtimeState</code> （已解析）	mm	deg

重要: CRI UDP 二进制载荷使用米和弧度。SDK 在内部自动转换为 mm/deg。不要假设原始 UDP 浮点数是 mm/deg。

命名约定

所有公共方法使用 **PascalCase**，与 C# 和 Python SDK 保持一致。

```
robot.ConnectRemoteAndSwitchOn("192.168.8.136");
int di = robot.GetDi(0);
robot.MovJ(joints, 40, 100);
robot.SetDo(10, 1);
```

线程安全

- `GetRobotRealtimeState()` — 线程安全（返回副本）
- `SetCriDataReceived(callback)` — 线程安全
- 所有 TCP 方法 — 可从任意线程调用，但不要在同一 `CodroidClient` 上并发调用
- `CriRealtimeDispatcher::SendCommand` / `SendTrajectory` — 非线程安全

异常类型

异常	触发条件	来源
<code>CodroidCommandException</code>	控制器返回 <code>err</code> 字段	TCP 响应
<code>CodroidException</code>	通用运行时错误	SDK 内部
<code>std::runtime_error</code>	标准库异常	标准库

CodroidCommandException 属性

```
class CodroidCommandException : public CodroidException {
public:
    int request_id() const noexcept;           // 协议请求 ID
    const std::string& command_ty() const noexcept; // 如 "Robot/move"
    const std::string& controller_error() const noexcept; // 控制器的 err 字段
    const std::string& raw_response_json() const noexcept; // 完整响应
};
```

CodroidClient API 参考

类: `CodroidClient`
命名空间: `Codroid`
头文件: `Codroid/client.hpp`

构造与析构

CodroidClient()

默认构造函数，不建立连接。

```
Codroid::CodroidClient robot;
```

~CodroidClient()

析构函数。如果未调用 `Disconnect()`，析构时会自动清理资源。

连接管理

Connect

```
bool Connect(const std::string& ip, int port = 9001);
```

建立 TCP 连接，不切模式、不自动上电。

参数	类型	默认值	说明
<code>ip</code>	<code>string</code>	—	控制器 IP 地址
<code>port</code>	<code>int</code>	9001	TCP 端口

返回值: `bool` — 连接成功返回 `true`

```
if (!robot.Connect("192.168.8.136")) {  
    std::cerr << "连接失败" << std::endl;  
}
```

ConnectRemoteAndSwitchOn

```
bool ConnectRemoteAndSwitchOn(const std::string& ip, int port = 9001, std::string local_ip = {});
```

连接 TCP、切换远程模式、然后上电。推荐的一键初始化方法。`local_ip` 非空时用于 `StartCriDataPush` 绑定本机 UDP。

参数	类型	默认值	说明
<code>ip</code>	<code>string</code>	—	控制器 IP 地址
<code>port</code>	<code>int</code>	9001	TCP 端口
<code>local_ip</code>	<code>string</code>	空	本机 IP，用于 CRI UDP 绑定

返回值： `bool` — 连接、切换远程、上电均成功返回 `true`

```
if (!robot.ConnectRemoteAndSwitchOn("192.168.8.136", 9001, "192.168.8.150")) {
    std::cerr << "连接失败" << std::endl;
    return 1;
}
```

Disconnect

```
void Disconnect();
```

断开 TCP 连接，停止 CRI 相关线程与缓存。始终在程序结束前调用。

返回值： 无

```
try {
    robot.ConnectRemoteAndSwitchOn("192.168.8.136", 9001, "192.168.8.150");
    // ... 操作 ...
} catch (...) {
    robot.Disconnect();
    throw;
}
robot.Disconnect();
```

NextRequestId

```
int NextRequestId();
```

生成下一个单调递增的请求 ID。多线程发令时避免重复。

返回值： `int` — 下一个可用的请求 ID

模式控制

SwitchOn / SwitchOff

```
CommandResult SwitchOn(int id = 1);
CommandResult SwitchOff(int id = 1);
```

机器人上电 / 下电。

参数	类型	默认值	说明
id	int	1	请求 ID

返回值：CommandResult — 控制器响应

```
robot.SwitchOn();
// ... 操作 ...
robot.SwitchOff();
```

ToManual / ToAuto / ToRemote

```
CommandResult ToManual(int id = 1);
CommandResult ToAuto(int id = 1);
CommandResult ToRemote(int id = 1);
```

切换到手动/自动/远程模式。

参数	类型	默认值	说明
id	int	1	请求 ID

返回值：CommandResult — 控制器响应

```
robot.ToRemote();
```

EnterManualModeViaAuto / EnterRemoteModeViaAuto

```
CommandResult EnterManualModeViaAuto(int id = 1);
CommandResult EnterRemoteModeViaAuto(int id = 1);
```

先切自动再切手动/远程。满足控制器"必须经过自动模式"的限制。

参数	类型	默认值	说明
id	int	1	请求 ID

返回值： `CommandResult` — 控制器响应

```
robot.EnterRemoteModeViaAuto();
```

ToSimulation / ToActual

```
CommandResult ToSimulation(int id = 1);
CommandResult ToActual(int id = 1);
```

进入仿真/实机模式。

参数	类型	默认值	说明
id	int	1	请求 ID

返回值： `CommandResult` — 控制器响应

StartDrag / StopDrag

```
CommandResult StartDrag(int id = 1);
CommandResult StopDrag(int id = 1);
```

进入/退出拖拽示教模式。

参数	类型	默认值	说明
id	int	1	请求 ID

返回值： `CommandResult` — 控制器响应

ClearSystemError

```
CommandResult ClearSystemError(int id = 1);
```

清除系统错误状态。

参数	类型	默认值	说明
id	int	1	请求 ID

返回值： `CommandResult` — 控制器响应

IO 操作

GetDi / GetDo / GetAi / GetAo

```
int GetDi(int port, int id = 1);
int GetDo(int port, int id = 1);
double GetAi(int port, int id = 1);
double GetAo(int port, int id = 1);
```

读取数字量/模拟量输入/输出。GetDi / GetDo 返回 0 或 1；GetAi / GetAo 返回浮点值。

参数	类型	默认值	说明
port	int	—	IO 端口号
id	int	1	请求 ID

返回值：GetDi / GetDo → int (0 或 1)；GetAi / GetAo → double

```
int di0 = robot.GetDi(0);
std::cout << "DI 0 = " << di0 << std::endl;

double ai1 = robot.GetAi(1);
std::cout << "AI 1 = " << ai1 << std::endl;
```

SetDo / SetAo

```
CommandResult SetDo(int port, int value, int id = 1);
CommandResult SetAo(int port, double value, int id = 1);
```

设置数字量/模拟量输出。SetDo 的 value 必须为 0 或 1。

参数	类型	默认值	说明
port	int	—	IO 端口号
value	int / double	—	写入值（SetDo 为 0 或 1，SetAo 为浮点值）
id	int	1	请求 ID

返回值：CommandResult — 控制器响应

```
robot.SetDo(10, 1);    // 设为 ON
robot.SetDo(10, 0);    // 设为 OFF
robot.SetAo(0, 3.14);
```

寄存器操作

GetRegisterValue

```
double GetRegisterValue(int address, int id = 1);
```

读取单个寄存器值。

参数	类型	默认值	说明
address	int	—	寄存器地址
id	int	1	请求 ID

返回值：double — 寄存器值

```
double value = robot.GetRegisterValue(49100);
std::cout << "Register 49100 = " << value << std::endl;
```

GetRegisterValues

```
std::vector<ClientRegisterInfo> GetRegisterValues(const std::vector<int>& addresses, int id = 1);
```

批量读取寄存器值。

参数	类型	默认值	说明
addresses	vector<int>	—	寄存器地址列表
id	int	1	请求 ID

返回值：vector<ClientRegisterInfo> — 寄存器值列表，顺序与输入地址一致

```
std::vector<int> addresses = {49100, 49101, 49102};
auto values = robot.GetRegisterValues(addresses);
for (auto& v : values)
    std::cout << "Address " << v.address << " = " << v.value << std::endl;
```

SetRegisterValue

```
CommandResult SetRegisterValue(int address, double value, int id = 1);
```

设置寄存器值。

参数	类型	默认值	说明
address	int	—	寄存器地址
value	double	—	写入值
id	int	1	请求 ID

返回值： `CommandResult` — 控制器响应

```
robot.SetRegisterValue(49100, 42);
robot.SetRegisterValue(49101, 3.14);
```

运动控制（非阻塞）

所有运动方法发送指令后立即返回。使用 `*Sync` 变体进行阻塞等待。

MovJ -- 关节运动

```
CommandResult MovJ(const ClientJointPoint& target, double speed, double acceleration,
                  double blend = -1, double relativeBlend = -1,
                  const std::vector<double>& coor = {}, const std::vector<double>& tool =
                  {}, int id = 1);

CommandResult MovJ(const ClientCartesianPoint& target, double speed, double acceleration,
                  double blend = -1, double relativeBlend = -1,
                  const std::vector<double>& coor = {}, const std::vector<double>& tool =
                  {}, int id = 1);
```

参数	类型	默认值	说明
target	ClientJointPoint / ClientCartesianPoint	—	目标位置
speed	double	—	速度（deg/s）
acceleration	double	—	加速度
blend	double	-1	平滑半径。与 <code>relativeBlend</code> 互斥——同时传入时 <code>relativeBlend</code> 无效。-1 表示无过渡
relativeBlend	double	-1	相对平滑比（0-100）。与 <code>blend</code> 互斥——同时传入时此参数无效
coor	vector<double>	{}	用户坐标系。空时不包含该字段
tool	vector<double>	{}	工具坐标系。空时不包含该字段

参数	类型	默认值	说明
<code>id</code>	<code>int</code>	1	请求 ID

返回值： `CommandResult` — 控制器响应

```
// 关节目标
auto joints = Codroid::JointPoint::Degrees({0, 0, 90, 0, 90, 0});
robot.MovJ(joints, 40, 100);

// 笛卡尔目标（关节运动到 TCP 位姿）
auto pose = Codroid::CartesianPoint::MmDeg({400, 0, 300, 180, 0, 0});
robot.MovJ(pose, 40, 100);
```

MovL -- 直线运动

```
CommandResult MovL(const ClientCartesianPoint& target, double speed, double acceleration,
                  double blend = -1, double relativeBlend = -1,
                  const std::vector<double>& coor = {}, const std::vector<double>& tool =
                  {}, int id = 1);

CommandResult MovL(const ClientJointPoint& target, double speed, double acceleration,
                  double blend = -1, double relativeBlend = -1,
                  const std::vector<double>& coor = {}, const std::vector<double>& tool =
                  {}, int id = 1);
```

参数	类型	默认值	说明
<code>target</code>	<code>ClientCartesianPoint</code> / <code>ClientJointPoint</code>	—	目标位置
<code>speed</code>	<code>double</code>	—	速度（mm/s）
<code>acceleration</code>	<code>double</code>	—	加速度
<code>blend</code>	<code>double</code>	-1	平滑半径。与 <code>relativeBlend</code> 互斥——同时传入时 <code>relativeBlend</code> 无效。-1 表示无过渡
<code>relativeBlend</code>	<code>double</code>	-1	相对平滑比（0-100）。与 <code>blend</code> 互斥——同时传入时此参数无效
<code>coor</code>	<code>vector<double></code>	{}	用户坐标系。空时不包含该字段
<code>tool</code>	<code>vector<double></code>	{}	工具坐标系。空时不包含该字段

参数	类型	默认值	说明
id	int	1	请求 ID

返回值： `CommandResult` — 控制器响应

```
auto state = robot.GetRobotRealtimeState();
auto target = Codroid::CartesianPoint::MmDegWithRef(
    {400, 0, 300, 180, 0, 0}, state.joint_position);
robot.MovL(target, 150, 500);
```

MovC -- 圆弧运动

```
CommandResult MovC(const ClientCartesianPoint& middle, const ClientCartesianPoint& target,
    double speed, double acceleration,
    double blend = -1, double relativeBlend = -1,
    const std::vector<double>& coor = {}, const std::vector<double>& tool =
    {}, int id = 1);
```

参数	类型	默认值	说明
middle	ClientCartesianPoint	—	中间点（圆弧上）
target	ClientCartesianPoint	—	终点
speed	double	—	速度（mm/s）
acceleration	double	—	加速度
blend	double	-1	平滑半径。与 <code>relativeBlend</code> 互斥——同时传入时 <code>relativeBlend</code> 无效。-1 表示无过渡
relativeBlend	double	-1	相对平滑比（0-100）。与 <code>blend</code> 互斥——同时传入时此参数无效
coor	vector<double>	{}	用户坐标系。空时不包含该字段
tool	vector<double>	{}	工具坐标系。空时不包含该字段
id	int	1	请求 ID

返回值： `CommandResult` — 控制器响应

```
auto mid = Codroid::CartesianPoint::MmDeg({450, 100, 300, 180, 0, 0});
auto end = Codroid::CartesianPoint::MmDeg({500, 0, 300, 180, 0, 0});
robot.MovC(mid, end, 100, 300);
```

MovCircle -- 整圆运动

```
CommandResult MovCircle(const ClientCartesianPoint& middle, const ClientCartesianPoint&
target,
                        int circle_num, double speed, double acceleration,
                        double blend = -1, double relativeBlend = -1,
                        const std::vector<double>& coor = {}, const std::vector<double>&
tool = {}, int id = 1);
```

参数	类型	默认值	说明
middle	ClientCartesianPoint	—	中间点
target	ClientCartesianPoint	—	终点
circle_num	int	—	整圆圈数
speed	double	—	速度 (mm/s)
acceleration	double	—	加速度
blend	double	-1	平滑半径。与 relativeBlend 互斥——同时传入时 relativeBlend 无效。-1 表示无过渡
relativeBlend	double	-1	相对平滑比 (0-100)。与 blend 互斥——同时传入时此参数无效
coor	vector<double>	{}	用户坐标系。空时不包含该字段
tool	vector<double>	{}	工具坐标系。空时不包含该字段
id	int	1	请求 ID

返回值： CommandResult — 控制器响应

```
auto mid = Codroid::CartesianPoint::MmDeg({450, 100, 300, 180, 0, 0});
auto end = Codroid::CartesianPoint::MmDeg({500, 0, 300, 180, 0, 0});
robot.MovCircle(mid, end, 1, 80, 200);
```

Move / MovePath -- 多段路径

```
CommandResult Move(const std::vector<ClientMoveInstruction>& path, int id = 1);
CommandResult MovePath(const std::vector<ClientMoveInstruction>& path, int id = 1);
```

将一组运动指令作为单条路径命令发送。MovePath 为 Move 的别名。

参数	类型	默认值	说明
path	vector<ClientMoveInstruction>	—	运动指令列表
id	int	1	请求 ID

返回值： CommandResult — 控制器响应

```
auto state = robot.GetRobotRealtimeState();
std::vector<Codroid::ClientMoveInstruction> path = {
    Codroid::ClientMoveInstruction::MovJ(
        Codroid::JointPoint::Degrees({0, 0, 90, 0, 90, 0}), 40, 100),
    Codroid::ClientMoveInstruction::MovL(
        Codroid::CartesianPoint::MmDegWithRef({400, 0, 300, 180, 0, 0},
state.joint_position),
        150, 500)
};
robot.Move(path);
```

阻塞运动（Sync）

*Sync 方法发送运动指令后阻塞直到 CRI 确认机器人到达目标。成功返回 true，错误/超时抛出异常。

必须先调用 StartCriDataPush 并等待首帧（WaitForCriData）。

MoveSync

```
bool MoveSync(const std::vector<ClientMoveInstruction>& path, const MotionWaitOptions& wait = {});
```

发送多段路径并阻塞直到最后一段目标到达。

参数	类型	默认值	说明
path	vector<ClientMoveInstruction>	—	运动指令列表
wait	MotionWaitOptions	默认值	等待选项（超时、容差等）

返回值： bool — 成功到达目标返回 true

异常： 运动超时由 MotionWaitOptions.timeout_s 控制；机器人异常状态（碰撞、急停、报警）时返回 false

```
robot.MoveSync({
    Codroid::ClientMoveInstruction::MovJ(
        Codroid::JointPoint::Degrees({0, 0, 90, 0, 90, 0}), 40, 100),
    Codroid::ClientMoveInstruction::MovL(target, 150, 500)
});
```

MovJSync

```
bool MovJSync(const ClientJointPoint& target, double speed, double acc,
              const MotionWaitOptions& wait = {},
              double blend = -1, double relativeBlend = -1,
              const std::vector<double>& coor = {}, const std::vector<double>& tool = {});

bool MovJSync(const ClientCartesianPoint& target, double speed, double acc,
              const MotionWaitOptions& wait = {},
              double blend = -1, double relativeBlend = -1,
              const std::vector<double>& coor = {}, const std::vector<double>& tool = {});
```

参数	类型	默认值	说明
target	ClientJointPoint / ClientCartesianPoint	—	目标位置
speed	double	—	速度（deg/s）
acc	double	—	加速度
wait	MotionWaitOptions	默认值	等待选项（超时、容差等）
blend	double	-1	平滑半径。与 relativeBlend 互斥——同时传入时 relativeBlend 无效。-1 表示无过渡
relativeBlend	double	-1	相对平滑比（0-100）。与 blend 互斥——同时传入时此参数无效
coor	vector<double>	{}	用户坐标系。空时不包含该字段
tool	vector<double>	{}	工具坐标系。空时不包含该字段

返回值： bool — 成功到达目标返回 true

异常： 机器人异常状态（碰撞、急停、报警）时返回 false

```
Codroid::MotionWaitOptions wait;
wait.timeout_s = 90.0;
wait.joint_tolerance_deg = 0.3;

robot.MovJSync(Codroid::JointPoint::Degrees({0, 0, 90, 0, 90, 0}), 40, 100, wait);
```

MovLSync

```
bool MovLSync(const ClientCartesianPoint& target, double speed, double acc,
              const MotionWaitOptions& wait = {},
              double blend = -1, double relativeBlend = -1,
              const std::vector<double>& coor = {}, const std::vector<double>& tool = {});

bool MovLSync(const ClientJointPoint& target, double speed, double acc,
              const MotionWaitOptions& wait = {},
              double blend = -1, double relativeBlend = -1,
              const std::vector<double>& coor = {}, const std::vector<double>& tool = {});
```

参数	类型	默认值	说明
target	ClientCartesianPoint / ClientJointPoint	—	目标位置
speed	double	—	速度（mm/s）
acc	double	—	加速度
wait	MotionWaitOptions	默认值	等待选项
blend	double	-1	平滑半径。与 relativeBlend 互斥——同时传入时 relativeBlend 无效。-1 表示无过渡
relativeBlend	double	-1	相对平滑比（0-100）。与 blend 互斥——同时传入时此参数无效
coor	vector<double>	{}	用户坐标系。空时不包含该字段
tool	vector<double>	{}	工具坐标系。空时不包含该字段

返回值： bool — 成功到达目标返回 true

异常： 机器人异常状态（碰撞、急停、报警）时返回 false

```
auto state = robot.GetRobotRealtimeState();
auto target = Codroid::CartesianPoint::MmDegWithRef(
    {400, 0, 300, 180, 0, 0}, state.joint_position);

Codroid::MotionWaitOptions wait;
wait.timeout_s = 60.0;
wait.cartesian_position_tolerance_mm = 2.0;
wait.cartesian_orientation_tolerance_deg = 1.5;

robot.MovLSync(target, 150, 500, wait);
```

MovCSync

```
bool MovCSync(const ClientCartesianPoint& middle, const ClientCartesianPoint& target,
    double speed, double acc, const MotionWaitOptions& wait = {},
    double blend = -1, double relativeBlend = -1,
    const std::vector<double>& coor = {}, const std::vector<double>& tool = {});
```

参数	类型	默认值	说明
middle	ClientCartesianPoint	—	中间点（圆弧上）
target	ClientCartesianPoint	—	终点
speed	double	—	速度（mm/s）
acc	double	—	加速度
wait	MotionWaitOptions	默认值	等待选项
blend	double	-1	平滑半径。与 relativeBlend 互斥——同时传入时 relativeBlend 无效。-1 表示无过渡
relativeBlend	double	-1	相对平滑比（0-100）。与 blend 互斥——同时传入时此参数无效
coor	vector<double>	{}	用户坐标系。空时不包含该字段
tool	vector<double>	{}	工具坐标系。空时不包含该字段

返回值： bool — 成功到达目标返回 true

异常： 机器人异常状态（碰撞、急停、报警）时返回 false

```
auto mid = Codroid::CartesianPoint::MmDeg({450, 100, 300, 180, 0, 0});
auto end = Codroid::CartesianPoint::MmDeg({500, 0, 300, 180, 0, 0});
robot.MovCSync(mid, end, 100, 300);
```

MovCircleSync

```
bool MovCircleSync(const ClientCartesianPoint& middle, const ClientCartesianPoint& target,
    int circle_num, double speed, double acc, const MotionWaitOptions& wait = {},
    double blend = -1, double relativeBlend = -1,
    const std::vector<double>& coor = {}, const std::vector<double>& tool = {});
```

参数	类型	默认值	说明
middle	ClientCartesianPoint	—	中间点
target	ClientCartesianPoint	—	终点
circle_num	int	—	整圆圈数
speed	double	—	速度（mm/s）
acc	double	—	加速度
wait	MotionWaitOptions	默认值	等待选项
blend	double	-1	平滑半径。与 relativeBlend 互斥——同时传入时 relativeBlend 无效。-1 表示无过渡
relativeBlend	double	-1	相对平滑比（0-100）。与 blend 互斥——同时传入时此参数无效
coor	vector<double>	{}	用户坐标系。空时不包含该字段
tool	vector<double>	{}	工具坐标系。空时不包含该字段

返回值： bool — 成功到达目标返回 true

异常： 机器人异常状态（碰撞、急停、报警）时返回 false

运动控制

PauseRobotMotion

```
CommandResult PauseRobotMotion(int id = 1);
```

暂停当前运动。

参数	类型	默认值	说明
id	int	1	请求 ID

返回值： CommandResult — 控制器响应

ResumeRobotMotion

```
CommandResult ResumeRobotMotion(int id = 1);
```

恢复暂停的运动。

参数	类型	默认值	说明
id	int	1	请求 ID

返回值： CommandResult — 控制器响应

StopRobotMove

```
CommandResult StopRobotMove(int id = 1);
```

立即停止当前运动。

参数	类型	默认值	说明
id	int	1	请求 ID

返回值： CommandResult — 控制器响应

MoveTo（规划运动）

MoveTo

```
CommandResult MoveTo(const MoveToParams& params, int id = 1);
```

移动到预设或规划位置。使用 `MoveToType::Joint` 或 `Line` 时需要心跳。

参数	类型	默认值	说明
params	MoveToParams	—	目标类型与可选目标点
id	int	1	请求 ID

返回值： `CommandResult` — 控制器响应

```
// 移动到原点
robot.MoveTo(Codroid::MoveToParams{Codroid::MoveToType::Home});

// 移动到安全位
robot.MoveTo(Codroid::MoveToParams{Codroid::MoveToType::Safe});
```

MoveToHeartbeat

```
CommandResult MoveToHeartbeat(int id = 1);
```

发送心跳以维持 MoveTo 运动。使用 `MoveToType::Joint` 或 `Line` 时每 $\geq 500\text{ms}$ 调用一次。

参数	类型	默认值	说明
id	int	1	请求 ID

返回值： `CommandResult` — 控制器响应

StopMoveTo

```
CommandResult StopMoveTo(int id = 1);
```

停止当前 MoveTo 运动。

参数	类型	默认值	说明
id	int	1	请求 ID

返回值： `CommandResult` — 控制器响应

Jog（点动）

Jog

```
CommandResult Jog(const JogParams& params, int id = 1);
```

启动点动。需要约每 500ms 发送心跳。

参数	类型	默认值	说明
params	JogParams	—	点动参数（模式、速度、轴索引、坐标系）
id	int	1	请求 ID

返回值：CommandResult — 控制器响应

```
Codroid::JogParams jog;
jog.mode = Codroid::JogMode::Joint;
jog.speed = 10.0;
jog.index = 0;
jog.coorType = Codroid::CoorType::User;
jog.coorId = 0;
robot.Jog(jog);

// 持续发送心跳
while (jogging) {
    std::this_thread::sleep_for(std::chrono::milliseconds(500));
    robot.JogHeartbeat();
}
robot.StopJog();
```

StopJog

```
CommandResult StopJog(int id = 1);
```

停止点动。

参数	类型	默认值	说明
id	int	1	请求 ID

返回值：CommandResult — 控制器响应

JogHeartbeat

```
CommandResult JogHeartbeat(int id = 1);
```

发送心跳以维持点动状态。约每 500ms 调用一次。

参数	类型	默认值	说明
id	int	1	请求 ID

返回值： CommandResult — 控制器响应

CRI 实时数据

StartCriDataPush

```
CommandResult StartCriDataPush(const std::string& udpIp, int udpPort, int id = 1);
```

启动本地 UDP 监听并请求控制器推送 CRI 实时数据。

参数	类型	默认值	说明
udpIp	string	—	本地 IP 地址，用于接收 UDP 数据
udpPort	int	—	本地端口号
id	int	1	请求 ID

返回值： CommandResult — 控制器响应

```
robot.StartCriDataPush("192.168.8.150", 18888);
robot.WaitForCriData(5.0); // 等待首帧

robot.SetCriDataReceived([](const Codroid::ClientRealtimeState& data) {
    std::cout << "Joints: ";
    for (auto j : data.joint_position) std::cout << j << " ";
    std::cout << std::endl;
});
```

StopCriDataPush

```
CommandResult StopCriDataPush(int id = 1);
CommandResult StopCriDataPush(const std::string& udpIp, int udpPort, int id = 1);
```

请求控制器停止 CRI 数据推送并关闭本地 UDP 监听。

参数	类型	默认值	说明
udpIp	string	—	本地 IP 地址（可选重载）
udpPort	int	—	本地端口号（可选重载）
id	int	1	请求 ID

返回值： `CommandResult` — 控制器响应

WaitForCriData

```
void WaitForCriData(double timeout_s = 5.0);
```

阻塞等待第一个 CRI 数据帧到达。

参数	类型	默认值	说明
timeout_s	double	5.0	最大等待秒数

返回值： 无

CRI 实时控制

StartCriControl

```
CommandResult StartCriControl(int filterType, int durationMs, int startBuffer, int id = 1);
```

启用 CRI 实时控制模式。

参数	类型	默认值	说明
filterType	int	—	0=关闭，1=均值，2=二阶低通，3=椭圆
durationMs	int	—	控制周期（1~16ms，必须整除 1000）
startBuffer	int	—	起始缓冲帧数（1~100）
id	int	1	请求 ID

返回值： `CommandResult` — 控制器响应

```
robot.StartCriControl(1, 4, 5);
```

StopCriControl

```
CommandResult StopCriControl(int id = 1);
```

禁用 CRI 实时控制模式。

参数	类型	默认值	说明
id	int	1	请求 ID

返回值： CommandResult — 控制器响应

GetRobotRealtimeState

```
ClientRealtimeState GetRobotRealtimeState() const;
```

获取线程安全的 CRI 数据快照。

返回值： ClientRealtimeState — CRI 实时数据副本

```
auto state = robot.GetRobotRealtimeState();
std::cout << "J1 = " << state.joint_position[0] << " deg" << std::endl;
std::cout << "TCP X = " << state.tcp_pose[0] << " mm" << std::endl;
```

SetCriDataReceived

```
void SetCriDataReceived(std::function<void(const ClientRealtimeState&)> cb);
```

设置 CRI 数据回调。每次收到有效 CRI UDP 帧时触发。

参 数	类型	默认 值	说明
cb	function<void(const ClientRealtimeState&)>	—	回调函数；在内部接收线程上执行，避免长时间阻塞

返回值： 无

```
robot.SetCriDataReceived([](const Codroid::ClientRealtimeState& data) {
    if (data.has_alarm) {
        std::cerr << "机器人有报警!" << std::endl;
    }
});
```

GetCriUdpListenPort

```
int GetCriUdpListenPort() const;
```

获取本机 CRI 推送绑定的 UDP 监听端口。未启动推送时可能为 0。

返回值： `int` — UDP 监听端口

项目执行

EnterRemoteScriptMode

```
CommandResult EnterRemoteScriptMode(int id = 1);
```

请求进入远程脚本模式。

参数	类型	默认值	说明
<code>id</code>	<code>int</code>	1	请求 ID

返回值： `CommandResult` — 控制器响应

RunScript

```
CommandResult RunScript(const std::string& mainScript,
                        const std::unordered_map<std::string, std::string>& subThreads = {},
                        const std::unordered_map<std::string, std::string>& subPrograms = {},
                        const std::unordered_map<std::string, std::string>& interrupts = {},
                        const nlohmann::json& vars = {},
                        int id = 1);
```

发送脚本立即执行。

参数	类型	默认值	说明
<code>mainScript</code>	<code>string</code>	—	主脚本内容
<code>subThreads</code>	<code>unordered_map<string, string></code>	<code>{}</code>	子线程脚本
<code>subPrograms</code>	<code>unordered_map<string, string></code>	<code>{}</code>	子程序脚本
<code>interrupts</code>	<code>unordered_map<string, string></code>	<code>{}</code>	中断处理脚本
<code>vars</code>	<code>nlohmann::json</code>	<code>{}</code>	注入变量
<code>id</code>	<code>int</code>	1	请求 ID

返回值： `CommandResult` — 控制器响应

```
robot.RunScript("movej(j1, v50) sub1() end");
```

Run / RunByIndex / RunStep

```
CommandResult Run(const std::string& projectId, int id = 1);
CommandResult RunByIndex(int index, int id = 1);
CommandResult RunStep(const std::string& projectId, int id = 1);
```

按 ID / 索引启动项目 / 单步执行。

Run / RunStep 参数：

参数	类型	默认值	说明
<code>projectId</code>	<code>string</code>	—	项目 ID
<code>id</code>	<code>int</code>	1	请求 ID

RunByIndex 参数：

参数	类型	默认值	说明
<code>index</code>	<code>int</code>	—	项目索引
<code>id</code>	<code>int</code>	1	请求 ID

返回值： `CommandResult` — 控制器响应

```
robot.Run("project_001");
robot.RunByIndex(0);
robot.RunStep("project_001");
```

PauseProject / ResumeProject / StopProject

```
CommandResult PauseProject(int id = 1);
CommandResult ResumeProject(int id = 1);
CommandResult StopProject(int id = 1);
```

暂停、恢复、停止工程。

参数	类型	默认值	说明
<code>id</code>	<code>int</code>	1	请求 ID

返回值： `CommandResult` — 控制器响应

发布订阅

SubscribePublishTopic

```
ClientPublishSubscription SubscribePublishTopic(
    std::string topicTy,
    std::function<void(const ClientPublishNotification&)> handler,
    int tc_milliseconds = 100);
```

订阅 TCP 主题推送。返回可释放的订阅句柄。

参数	类型	默认值	说明
topicTy	string	—	主题名，如 PublishTopics::RobotStatus
handler	function<void(const ClientPublishNotification&)>	—	处理通知的回调；不应长时间阻塞
tcMilliseconds	int	100	推送周期（ms）

返回值： ClientPublishSubscription — 可释放的订阅句柄

```
auto sub = robot.SubscribePublishTopic(
    Codroid::PublishTopics::RobotStatus,
    [](const Codroid::ClientPublishNotification& n) {
        std::cout << "状态: " << n.db_json << std::endl;
    });

// 订阅在 sub.Dispose() 或析构前有效
std::this_thread::sleep_for(std::chrono::seconds(10));
sub.Dispose();
```

全局变量

GetGlobalVars

```
nlohmann::json GetGlobalVars(int id = 1);
```

获取所有全局变量。

参数	类型	默认值	说明
id	int	1	请求 ID

返回值： nlohmann::json — 全局变量 JSON

```
auto vars = robot.GetGlobalVars();
std::cout << vars.dump(2) << std::endl;
```

SaveGlobalVars

```
CommandResult SaveGlobalVars(const std::map<std::string, Variable>& vars, int id = 1);
```

保存全局变量。

参数	类型	默认值	说明
vars	map<string, Variable>	—	要保存的变量键值对
id	int	1	请求 ID

返回值： CommandResult — 控制器响应

```
robot.SaveGlobalVars({
    {"counter", Codroid::Variable(42, "计数器")},
    {"name", Codroid::Variable(std::string("test"), "名称")}
});
```

RemoveGlobalVars

```
CommandResult RemoveGlobalVars(const std::vector<std::string>& names, int id = 1);
```

删除指定全局变量。删除不存在的变量不会报错。

参数	类型	默认值	说明
names	vector<string>	—	要删除的变量名列表
id	int	1	请求 ID

返回值： CommandResult — 控制器响应

```
robot.RemoveGlobalVars({"counter", "name"});
```

运动学

ForwardKinematics

```
std::vector<double> ForwardKinematics(const FKParams& params, int id = 1);
```

正运动学：关节空间 -> 笛卡尔空间。返回 [x,y,z,rx,ry,rz]，单位 mm + deg。

参数	类型	默认值	说明
params	FKParams	—	关节角、坐标系、工具等参数
id	int	1	请求 ID

返回值：vector<double> — 笛卡尔位姿 [x,y,z,rx,ry,rz]

```
Codroid::FKParams fk({0, 0, 90, 0, 90, 0});
auto tcp = robot.ForwardKinematics(fk);
```

InverseKinematics

```
std::vector<double> InverseKinematics(const IKParams& params, int id = 1);
```

逆运动学：笛卡尔 -> 关节空间。返回 6 个关节角度（度）。

参数	类型	默认值	说明
params	IKParams	—	TCP 位姿、参考关节、外部轴等参数
id	int	1	请求 ID

返回值：vector<double> — 6 个关节角度（度）

```
Codroid::IKParams ik({400, 0, 300, 180, 0, 0});
ik.rj = {0, 0, 90, 0, 90, 0};
auto joints = robot.InverseKinematics(ik);
```

CalculateRelativePose

```
std::vector<double> CalculateRelativePose(const RelativePoseParams& params, int id = 1);
```

在用户或工具坐标系中计算相对位姿/偏移。

参数	类型	默认值	说明
params	RelativePoseParams	—	当前位姿、偏移量、坐标系类型等参数
id	int	1	请求 ID

返回值：vector<double> — 计算后的位姿 [x,y,z,rx,ry,rz]

```
Codroid::RelativePoseParams rp;
rp.pos = {400, 0, 300, 180, 0, 0};
rp.offset = {50, 0, 0, 0, 0, 0};
rp.coorType = Codroid::CoorType::User;
auto newPose = robot.CalculateRelativePose(rp);
```

机器人设置参数

GetRobotParameters

```
ClientRobotParameters GetRobotParameters(int id = 1);
```

获取所有设置界面参数。返回工具坐标系、载荷坐标系、用户坐标系及默认 ID。

参数	类型	默认值	说明
id	int	1	请求 ID

返回值：ClientRobotParameters — 机器人完整参数集（valid 为 false 表示获取失败）

```
auto param = robot.GetRobotParameters();
if (param.valid) {
    std::cout << "默认工具: " << param.default_tool_id << std::endl;
    std::cout << "最大载荷: " << param.max_payload << " kg" << std::endl;
}
```

SetDefaultPayloadId / SetDefaultToolId / SetDefaultUserCoordinateId

```
CommandResult SetDefaultPayloadId(int payloadId, int id = 1);
CommandResult SetDefaultToolId(int toolId, int id = 1);
CommandResult SetDefaultUserCoordinateId(int coordinateId, int id = 1);
```

设置默认载荷/工具/用户坐标系编号（1~15）。

参数	类型	默认值	说明
payloadId / toolId / coordinateId	int	—	槽位编号，范围 1~15

参数	类型	默认值	说明
id	int	1	请求 ID

返回值： `CommandResult` — 控制器响应

```
robot.SetDefaultToolId(2);
robot.SetDefaultPayloadId(1);
robot.SetDefaultUserCoordinateId(0);
```

SaveToolFrames / SetToolFrame

```
CommandResult SaveToolFrames(const std::vector<ClientRobotFrame>& frames, int id = 1);
CommandResult SetToolFrame(int frame_id, const ClientRobotFrame& frame, int id = 1);
```

保存完整工具坐标系表 / 修改单个工具坐标系（`frame_id` 为 1~15）。

SaveToolFrames 参数：

参数	类型	默认值	说明
frames	vector<ClientRobotFrame>	—	工具坐标系列表
id	int	1	请求 ID

SetToolFrame 参数：

参数	类型	默认值	说明
frame_id	int	—	工具坐标系编号，范围 1~15
frame	ClientRobotFrame	—	坐标系定义
id	int	1	请求 ID

返回值： `CommandResult` — 控制器响应

```
Codroid::CodroidClient::ClientRobotFrame tool;
tool.id = 1; tool.x = 0; tool.y = 0; tool.z = 100;
tool.a = 0; tool.b = 0; tool.c = 0;
robot.SetToolFrame(1, tool);
```

SavePayloadFrames / SetPayloadFrame

```
CommandResult SavePayloadFrames(const std::vector<ClientRobotPayload>& frames, int id = 1);
CommandResult SetPayloadFrame(int frame_id, const ClientRobotPayload& frame, int id = 1);
```

保存完整载荷坐标系表 / 修改单个载荷坐标系（frame_id 为 1~15）。

SavePayloadFrames 参数：

参数	类型	默认值	说明
frames	vector<ClientRobotPayload>	—	载荷坐标系列表
id	int	1	请求 ID

SetPayloadFrame 参数：

参数	类型	默认值	说明
frame_id	int	—	载荷坐标系编号，范围 1~15
frame	ClientRobotPayload	—	载荷坐标系定义
id	int	1	请求 ID

返回值：CommandResult — 控制器响应

SetUserCoordinateFrame

```
CommandResult SetUserCoordinateFrame(int frame_id, const ClientRobotFrame& frame, int id = 1);
```

修改单个用户坐标系（先读后改；frame_id 为 1~15）。

参数	类型	默认值	说明
frame_id	int	—	用户坐标系编号，范围 1~15
frame	ClientRobotFrame	—	坐标系定义
id	int	1	请求 ID

返回值：CommandResult — 控制器响应

SetPayload / SetManualMoveRate / SetAutoMoveRate / SetCollisionSensitivity

```
CommandResult SetPayload(int payloadId, int id = 1);
CommandResult SetManualMoveRate(int pct, int id = 1);
CommandResult SetAutoMoveRate(int pct, int id = 1);
CommandResult SetCollisionSensitivity(int sensitivity, int id = 1);
```

运行时参数设置。

SetPayload 参数：

参数	类型	默认值	说明
payloadId	int	—	载荷槽位编号（0~15）
id	int	1	请求 ID

SetManualMoveRate / SetAutoMoveRate 参数：

参数	类型	默认值	说明
pct	int	—	运动倍率百分比，范围 1~100
id	int	1	请求 ID

SetCollisionSensitivity 参数：

参数	类型	默认值	说明
sensitivity	int	—	碰撞检测灵敏度，范围 0~100
id	int	1	请求 ID

返回值： CommandResult — 控制器响应

```
robot.SetPayload(1);
robot.SetManualMoveRate(50); // 50% 速度
robot.SetAutoMoveRate(100); // 全速
robot.SetCollisionSensitivity(50);
```

错误处理设置

SetThrowOnCommandError / ThrowOnCommandError

```
void SetThrowOnCommandError(bool enable);
bool ThrowOnCommandError() const noexcept;
```

设置/获取错误处理模式。设为 `true` 时，指令失败将抛出 `CodroidCommandException`。

参数	类型	默认值	说明
<code>enable</code>	<code>bool</code>	—	<code>true</code> 启用异常模式

返回值： `SetThrowOnCommandError` 无返回； `ThrowOnCommandError` 返回 `bool`

```
robot.SetThrowOnCommandError(true);
try {
    robot.SetDo(999, 1);
} catch (const Codroid::CodroidCommandException& ex) {
    std::cerr << "控制器错误: " << ex.controller_error() << std::endl;
}
```


运动 API 参考

本文档涵盖 CodroidCPP SDK 中所有与运动相关的类型，包括关节/笛卡尔点定义、运动指令、点动参数和运动等待选项。

点位类型

JointPoint

关节空间目标点（六轴角，单位：度）。

```
struct JointPoint {
    std::vector<double> jp; // 六轴关节角（度）

    static JointPoint Degrees(std::vector<double> joints_deg);
};
```

属性	类型	说明
jp	vector<double>	六轴关节角（度）

工厂方法	说明
JointPoint::Degrees(vector<double> joints_deg)	从 6 个关节角度（度）创建。数组必须恰好为长度 6。

使用示例：

```
auto joints = Codroid::JointPoint::Degrees({0, 0, 90, 0, 90, 0});
robot.MovJ(joints, 40, 100);
```

CartesianPoint

笛卡尔末端目标点（TCP 位姿：mm + 度）。

```
struct CartesianPoint {
    std::vector<double> cp; // [x, y, z, rx, ry, rz]
    std::vector<double> rj; // 逆解参考关节角（度）

    static CartesianPoint MmDeg(std::vector<double> pose_mm_deg);
    static CartesianPoint MmDegWithRef(std::vector<double> pose_mm_deg, std::vector<double> ref_joints_deg);
};
```

属性	类型	说明
cp	vector<double>	TCP 位姿 [x, y, z, rx, ry, rz] — 位置单位 mm，姿态单位度
rj	vector<double>	用于逆运动学的参考关节（6 个关节角度，度）。空时使用默认值

工厂方法	说明
CartesianPoint::MmDeg(vector<double> pose)	仅使用 TCP 位姿创建（使用默认参考关节）
CartesianPoint::MmDegWithRef(vector<double> pose, vector<double> refJoints)	使用 TCP 位姿和显式参考关节创建

使用示例：

```
// 不指定参考关节
auto pose1 = Codroid::CartesianPoint::MmDeg({400, 0, 300, 180, 0, 0});

// 指定参考关节（推荐）
auto state = robot.GetRobotRealtimeState();
auto pose2 = Codroid::CartesianPoint::MmDegWithRef(
    {400, 0, 300, 180, 0, 0},
    state.joint_position
);
```

MovePoint

路径段中的单个目标点（协议层）。

```
struct MovePoint {
    std::vector<double> jp;
    std::vector<double> cp;
    std::vector<double> rj;
    std::vector<double> ep;

    static MovePoint Joint(JointPoint joint);
    static MovePoint Cartesian(CartesianPoint cart);
};
```

属性	类型	说明
jp	vector<double>	关节角度（度），笛卡尔目标时空
cp	vector<double>	TCP 位姿（mm + 度），关关节目标时空
rj	vector<double>	用于逆运动学的参考关节
ep	vector<double>	外部轴

路径指令

ClientMoveInstruction

路径中的一段运动指令。

```
struct ClientMoveInstruction {
    ClientMoveType type = ClientMoveType::MovJ;
    double speed = 60.0;
    double acceleration = 150.0;
    double blend = -1.0;
    double relative_blend = -1.0;
    int circle_num = 1;
    ClientMovePoint target;
    ClientMovePoint middle;
    std::vector<double> coor;
    std::vector<double> tool;

    static ClientMoveInstruction MovJ(ClientJointPoint target, double speed, double
acceleration, double blend = -1.0);
    static ClientMoveInstruction MovJ(ClientCartesianPoint target, double speed, double
acceleration, double blend = -1.0);
    static ClientMoveInstruction MovL(ClientCartesianPoint target, double speed, double
acceleration, double blend = -1.0);
    static ClientMoveInstruction MovL(ClientJointPoint target, double speed, double
acceleration, double blend = -1.0);
    static ClientMoveInstruction MovC(ClientCartesianPoint middle, ClientCartesianPoint
target, ...);
    static ClientMoveInstruction MovCircle(ClientCartesianPoint middle, ClientCartesianPoint
target, int circle_num, ...);
};
```

属性	类型	默认值	说明
type	ClientMoveType	MovJ	运动类型
speed	double	60.0	速度（直线 mm/s，关节 deg/s）
acceleration	double	150.0	加速度
blend	double	-1.0	平滑半径。与 relative_blend 互斥——同时设置时 relative_blend 无效。-1 表示无过渡
relative_blend	double	-1.0	相对平滑比（0-100）。与 blend 互斥——同时设置时此属性无效
circle_num	int	1	整圆圈数（仅用于 MovCircle）
target	ClientMovePoint	—	本段的目标点
middle	ClientMovePoint	—	中间/经过点（MovC 和 MovCircle 必需）

属性	类型	默认值	说明
coord	vector<double>	{}	用户坐标系定义
tool	vector<double>	{}	工具坐标系定义

工厂方法参数参考：

参数	类型	默认值	说明
target	ClientJointPoint / ClientCartesianPoint	—	目标点
speed	double	—	速度（mm/s 或 deg/s）
acceleration	double	—	加速度
blend	double	-1.0	平滑半径。-1 表示无过渡

使用示例：

```
auto state = robot.GetRobotRealtimeState();

std::vector<Codroid::ClientMoveInstruction> path = {
    Codroid::ClientMoveInstruction::MovJ(
        Codroid::JointPoint::Degrees({0, 0, 90, 0, 90, 0}), 40, 100),
    Codroid::ClientMoveInstruction::MovL(
        Codroid::CartesianPoint::MmDegWithRef({400, 0, 300, 180, 0, 0},
state.joint_position),
        150, 500)
};
robot.Move(path);
```

运动类型枚举

ClientMoveType

```
enum class ClientMoveType {
    MovJ,           // 关节运动
    MovL,           // 直线（笛卡尔）
    MovC,           // 圆弧（经由中间点）
    MovCircle       // 整圆/多圈
};
```

名称	说明
MovJ	关节运动
MovL	直线（笛卡尔）运动

名称	说明
MovC	圆弧运动（经由中间点）
MovCircle	整圆/多圈运动

阻塞运动参数

MotionWaitOptions

配置 SDK 等待运动完成的方式。

```
struct MotionWaitOptions {
    double timeout_s = 60.0;
    double poll_interval_s = 0.05;
    double cri_stale_timeout_s = 0.5;
    int settled_samples = 3;
    double joint_tolerance_deg = 0.2;
    double cartesian_position_tolerance_mm = 1.0;
    double cartesian_orientation_tolerance_deg = 1.0;
};
```

属性	类型	默认值	说明
timeout_s	double	60.0	等待运动完成的最长时间（秒）
poll_interval_s	double	0.05	检查运动状态的轮询间隔（秒）
cri_stale_timeout_s	double	0.5	CRI 数据被视为过期的最长时间（秒）
settled_samples	int	3	确认运动完成所需的连续稳定采样数
joint_tolerance_deg	double	0.2	关节位置容差（度）
cartesian_position_tolerance_mm	double	1.0	笛卡尔位置容差（mm）
cartesian_orientation_tolerance_deg	double	1.0	笛卡尔姿态容差（度）

使用示例：

```
// 使用默认选项
robot.MovJSync(Codroid::JointPoint::Degrees({0, 0, 90, 0, 90, 0}), 40, 100);

// 自定义等待选项
Codroid::MotionWaitOptions wait;
wait.timeout_s = 120.0;
wait.poll_interval_s = 0.02;
wait.settled_samples = 5;
wait.joint_tolerance_deg = 0.05;
wait.cartesian_position_tolerance_mm = 0.2;
robot.MovLSync(target, 50, 200, wait);
```

MoveTo 参数

MoveToType

```
enum class MoveToType : int {
    Stop = -1,          // 停止 MoveTo
    Home = 0,           // 回 Home
    Safe = 1,           // 回安全位
    Candle = 2,         // 回 Candle 位
    Packing = 3,        // 回 Packing 位
    Joint = 4,          // 关节规划到目标
    Line = 5,           // 直线规划到目标
    ResumePoint = 6     // 恢复程序执行
};
```

名称	值	说明
Stop	-1	停止 MoveTo 操作
Home	0	原点位置
Safe	1	安全位置
Candle	2	烛台（垂直）位置
Packing	3	打包（运输）位置
Joint	4	关节规划运动到目标
Line	5	直线规划运动到目标
ResumePoint	6	恢复程序执行

MoveToParams

```
struct MoveToParams {
    MoveToType type = MoveToType::Home;
    MoveToTarget target;
};
```

属性	类型	说明
type	MoveToType	目标类型
target	MoveToTarget	规划目标点（仅 Joint/Line 时需要）

Jog 参数

JogMode

```
enum class JogMode {
    Joint = 1, // 关节点动
    Line = 2 // 直线点动
};
```

名称	值	说明
Joint	1	点动单个关节
Line	2	在笛卡尔空间中直线点动

JogParams

```
struct JogParams {
    JogMode mode = JogMode::Line;
    double speed = 0.0;
    int index = 1;
    CoordType coordType = CoordType::User;
    int coordId = 1;
};
```

属性	类型	默认值	说明
mode	JogMode	Line	点动模式：关节或直线
speed	double	0.0	点动速度（-1~1 的比例）
index	int	1	轴号或方向
coordType	CoordType	User	坐标系类型：用户或工具

属性	类型	默认值	说明
<code>coorId</code>	<code>int</code>	1	坐标系 ID

数据类型与枚举

通信相关

CommandResult

大多数 TCP 指令返回的结果。

```
struct CommandResult {
    int id = 0;
    std::string ty;
    std::string error_msg;
    std::string raw_json;

    bool Ok() const noexcept;
};
```

属性	类型	说明
id	int	请求 ID
ty	string	响应类型 / 路由
error_msg	string	错误信息（空表示成功）
raw_json	string	完整响应 JSON

方法	返回类型	说明
Ok()	bool	error_msg 为空返回 true

实时数据

ClientRealtimeState

CRI 实时数据快照，包含关节位置、TCP 位姿、状态标志等。

时间戳

属性	类型	说明
timestamp_ms	int64_t	控制器端的时间戳，单位为毫秒
data_valid	bool	数据是否有效

状态标志

属性	类型	说明
status1_raw	uint16_t	控制器原始状态寄存器 1
status2_raw	uint16_t	控制器原始状态寄存器 2
project_running	bool	当前是否有程序在运行
project_stopped	bool	程序是否已停止
project_paused	bool	程序是否处于暂停状态
enabling	bool	使能开关是否处于激活状态
not_enabled	bool	机器人未处于使能状态
manual_mode	bool	当前是否为手动操作模式
dragging	bool	机器人是否处于拖动示教状态
in_motion	bool	机器人当前是否有轴在运动
collision_stopped	bool	机器人是否因检测到碰撞而停止
in_safety_position	bool	机器人是否已到达安全位置
has_alarm	bool	控制器是否有活跃的报警信息
simulation_mode	bool	控制器是否运行在仿真模式下
emergency_stop_pressed	bool	急停按钮是否被按下
rescue_mode	bool	机器人是否处于碰撞救援模式
auto_mode	bool	控制器是否处于自动运行模式
remote_mode	bool	控制器是否处于远程控制模式
realtime_control_mode	bool	控制器是否处于实时控制模式
cri_error_code	uint8_t	CRI 协议层的错误代码

关节数据

属性	类型	说明
joint_position	vector<double>	各关节的当前角度，单位为度
joint_velocity	vector<double>	各关节的当前角速度
joint_output_torque	vector<double>	各关节的当前输出力矩百分比
joint_external_force	vector<double>	各关节检测到的外部力

TCP 数据

属性	类型	说明
tcp_pose	vector<double>	工具中心点位姿，XYZ 单位 mm，ABC 单位度
tcp_velocity	vector<double>	工具中心点的六维速度
tcp_linear_velocity_mm_s	double	工具中心点的线速度标量值

外部轴

属性	类型	说明
external_axis_position	vector<double>	外部轴（如导轨、转台）的位置数组

IO 相关

ClientIoInfo / ClientRegisterInfo

```
struct ClientIoInfo {
    std::string type;
    int port = 0;
    double value = 0.0;
};

struct ClientRegisterInfo {
    int address = 0;
    double value = 0.0;
};
```

属性	类型	说明
type	string	IO 类型字符串
port	int	端口号
value	double	值
address	int	寄存器地址

机器人设置

ClientRobotFrame / ClientRobotPayload

```
struct ClientRobotFrame {
    int id = 0;
    double x = 0.0, y = 0.0, z = 0.0;
    double a = 0.0, b = 0.0, c = 0.0;
};

struct ClientRobotPayload {
    int id = 0;
    double m = 0.0;
    double mx = 0.0, my = 0.0, mz = 0.0;
};
```

ClientRobotFrame 属性：

属性	类型	说明
id	int	坐标系的唯一编号
x, y, z	double	位置偏移（mm）
a, b, c	double	姿态角度（度）

ClientRobotPayload 属性：

属性	类型	说明
id	int	负载配置的唯一编号
m	double	负载质量（kg）
mx, my, mz	double	质心偏移（mm）

ClientRobotParameters

```
struct ClientRobotParameters {
    bool valid = false;
    int default_tool_id = 0;
    int default_payload_id = 0;
    int default_coordinate_id = 0;
    double max_payload = 0.0;
    std::vector<ClientRobotFrame> tool;
    std::vector<ClientRobotPayload> payload;
    std::vector<ClientRobotFrame> coordinate;
};
```

属性	类型	说明
<code>valid</code>	<code>bool</code>	数据是否有效（获取失败为 false）
<code>default_tool_id</code>	<code>int</code>	当前激活的工具坐标系编号
<code>default_payload_id</code>	<code>int</code>	当前激活的负载配置编号
<code>default_coordinate_id</code>	<code>int</code>	当前激活的用户坐标系编号
<code>max_payload</code>	<code>double</code>	机器人允许的最大负载质量（kg）
<code>tool</code>	<code>vector<ClientRobotFrame></code>	所有已配置的工具坐标系
<code>payload</code>	<code>vector<ClientRobotPayload></code>	所有已配置的负载参数
<code>coordinate</code>	<code>vector<ClientRobotFrame></code>	所有已配置的用户坐标系

主题推送

ClientPublishNotification

```
struct ClientPublishNotification {
    std::string ty;
    std::string db_json;
    std::string raw_json;
};
```

属性	类型	说明
<code>ty</code>	<code>string</code>	主题类型
<code>db_json</code>	<code>string</code>	业务载荷的 JSON 字符串
<code>raw_json</code>	<code>string</code>	完整 JSON 文本

ClientPublishSubscription

```
class ClientPublishSubscription {
public:
    void Dispose();
    bool IsValid() const noexcept;
};
```

方法	说明
<code>Dispose()</code>	提前结束订阅（与析构效果类似）
<code>IsValid()</code>	是否仍绑定有效内部资源

全局变量

Variable

```
struct Variable {
    std::string val;
    std::string nm;

    template<typename T>
    Variable(const T& value, const std::string& note = "");
};
```

属性	类型	说明
val	string	变量的JSON 值字符串
nm	string	变量的备注名称

运动学

FKParams / IKParams

```
struct FKParams {
    std::vector<double> jp;
    std::vector<double> coor, tool, ep;
};

struct IKParams {
    std::vector<double> cp;
    std::vector<double> rj;
    std::vector<double> ep;
};
```

FKParams 属性：

属性	类型	说明
jp	vector<double>	6 个关节角度（度）
coor	vector<double>	用户坐标系
tool	vector<double>	工具坐标系
ep	vector<double>	外部轴位置

IKParams 属性：

属性	类型	说明
cp	vector<double>	TCP 位姿 [x,y,z,rx,ry,rz] (mm + 度)
rj	vector<double>	参考关节角 (度)
ep	vector<double>	外部轴位置

RelativePoseParams

```
struct RelativePoseParams {
    std::vector<double> pos;
    std::vector<double> offset;
    CoordType coordType = CoordType::Tool;
    std::vector<double> posCoord;
    std::vector<double> coord;
};
```

属性	类型	说明
pos	vector<double>	世界坐标系中的当前 TCP 位姿
offset	vector<double>	[dx,dy,dz,drx,dry,drz] 偏移量
coordType	CoordType	用户或工具坐标系
posCoord	vector<double>	位置坐标系中的 TCP 位姿
coord	vector<double>	用户坐标系定义

异常类

CodroidException / CodroidCommandException

```
class CodroidException : public std::runtime_error {
public:
    explicit CodroidException(const std::string& message);
};

class CodroidCommandException : public CodroidException {
public:
    int request_id() const noexcept;
    const std::string& command_ty() const noexcept;
    const std::string& controller_error() const noexcept;
    const std::string& raw_response_json() const noexcept;
};
```

CodroidCommandException 属性：

属性	类型	说明
<code>request_id()</code>	<code>int</code>	用于匹配请求与响应的标识符
<code>command_ty()</code>	<code>string</code>	字符串标识符，表示哪个 CRI 命令失败
<code>controller_error()</code>	<code>string</code>	控制器返回的错误描述
<code>raw_response_json()</code>	<code>string</code>	完整响应 JSON，可用于进一步诊断

固件版本常量

```
inline constexpr const char* MinControllerFirmware = "2.3.3.43";
```


参数	类型	默认值	说明
<code>controller_ip</code>	<code>string</code>	—	机器人控制器的 IP 地址
<code>controller_udp_port</code>	<code>int</code>	9030	发送命令的 UDP 端口
<code>convert_to_si</code>	<code>bool</code>	<code>true</code>	若为 <code>true</code> ，发送前将度转换为弧度、毫米转换为米

SendCommand

```
void SendCommand(const std::array<double, 6>& position6, TrajectorySpace space);
```

参数	类型	说明
<code>position6</code>	<code>array<double, 6></code>	目标位置，必须为 6 个元素
<code>space</code>	<code>TrajectorySpace</code>	坐标空间： <code>Joint</code> 或 <code>Cartesian</code>

```
CriRealtimeDispatcher dispatcher("192.168.8.136");
dispatcher.SendCommand({0, 0, 90, 0, 90, 0}, TrajectorySpace::Joint);
```

SendTrajectory

```
void SendTrajectory(const std::vector<TrajectoryPoint>& trajectory, TrajectorySpace space,
int period_ms,
const std::atomic<bool>* cancel = nullptr);
```

参数	类型	说明
<code>trajectory</code>	<code>vector<TrajectoryPoint></code>	要发送的轨迹点序列
<code>space</code>	<code>TrajectorySpace</code>	坐标空间： <code>Joint</code> 或 <code>Cartesian</code>
<code>period_ms</code>	<code>int</code>	相邻点之间的时间间隔（毫秒）
<code>cancel</code>	<code>atomic<bool>*</code>	取消标志（可选）

```
CriRealtimeDispatcher dispatcher("192.168.8.136", 9030, true);
dispatcher.SendTrajectory(trajectory, TrajectorySpace::Joint, 4);
```

Close / IsOpen

```
void Close() noexcept;
bool IsOpen() const noexcept;
```

TrajectoryGenerator

离线轨迹生成。

```
#include "Codroid/trajjectory_generator.hpp"
```

Generate

```
static std::vector<TrajectoryPoint> Generate(const std::array<double, 6>& start,  
                                             const std::array<double, 6>& target,  
                                             const TrajectoryRequest& request);
```

参数	类型	说明
start	array<double, 6>	起始位置
target	array<double, 6>	目标位置
request	TrajectoryRequest	轨迹生成参数

返回值: `vector<TrajectoryPoint>` — 轨迹点序列

```
auto start = std::array<double, 6>{0, 0, 90, 0, 90, 0};
auto target = std::array<double, 6>{10, 20, 45, 0, 60, 30};

Codroid::TrajectoryRequest request;
request.space = Codroid::TrajectorySpace::Joint;
request.duration_s = 2.0;
request.frequency_hz = 250;

auto trajectory = Codroid::TrajectoryGenerator::Generate(start, target, request);
```

GenerateMultiSegment

```
static std::vector<TrajectoryPoint> GenerateMultiSegment(const  
std::vector<std::array<double, 6>>& waypoints,  
                                                         const TrajectoryRequest& request);
```

TrajectoryRequest

```
struct TrajectoryRequest {
    TrajectorySpace space = TrajectorySpace::Joint;
    TrajectoryProfile profile = TrajectoryProfile::Trapezoidal;
    double duration_s = 1.0;
    double frequency_hz = 250.0;
    double max_velocity = 0.0;
    double max_acceleration = 0.0;
    double max_jerk = 0.0;
};
```

属性	类型	默认值	说明
space	TrajectorySpace	Joint	轨迹的坐标空间
profile	TrajectoryProfile	Trapezoidal	运动曲线类型
duration_s	double	1.0	总时长（秒）
frequency_hz	double	250.0	采样频率（赫兹）
max_velocity	double	0.0	最大速度
max_acceleration	double	0.0	最大加速度
max_jerk	double	0.0	最大加加速度

TrajectorySpace / TrajectoryProfile

```
enum class TrajectorySpace { Joint, Cartesian };
enum class TrajectoryProfile { Cubic, Trapezoidal };
```

名称	说明
TrajectorySpace::Joint	关节空间：位置为关节角度
TrajectorySpace::Cartesian	笛卡尔空间：位置为工具位姿
TrajectoryProfile::Cubic	三次多项式曲线：平滑加减速
TrajectoryProfile::Trapezoidal	梯形速度曲线

完整示例

```
#include "Codroid/client.hpp"
#include "Codroid/cri_realtime_dispatcher.hpp"
#include "Codroid/trajectory_generator.hpp"

// 1. 连接
```

```
Codroid::CodroidClient robot;
robot.ConnectRemoteAndSwitchOn("192.168.8.136", 9001, "192.168.8.150");

// 2. 启动 CRI 推送
robot.StartCriDataPush("192.168.8.150", 9030);
robot.WaitForCriData(5.0);

// 3. 启动实时控制
robot.StartCriControl(1, 4, 5);

// 4. 等待实时控制模式生效
while (!robot.GetRobotRealtimeState().realtime_control_mode) {
    std::this_thread::sleep_for(std::chrono::milliseconds(10));
}

// 5. 生成轨迹
auto state = robot.GetRobotRealtimeState();
std::array<double, 6> start;
for (int i = 0; i < 6; i++) start[i] = state.joint_position[i];

Codroid::TrajectoryRequest request;
request.space = Codroid::TrajectorySpace::Joint;
request.duration_s = 2.0;
request.frequency_hz = 250;

auto trajectory = Codroid::TrajectoryGenerator::Generate(
    start, {10, 0, 90, 0, 90, 0}, request);

// 6. 下发轨迹
Codroid::CriRealtimeDispatcher dispatcher("192.168.8.136", 9030, true);
dispatcher.SendTrajectory(trajectory, Codroid::TrajectorySpace::Joint, 4);

// 7. 清理
robot.StopCriControl();
robot.StopCriDataPush();
robot.Disconnect();
```

IO 与寄存器 API 参考

数字量 IO

GetDi / GetDo

```
int GetDi(int port, int id = 1);
int GetDo(int port, int id = 1);
```

SetDo

```
CommandResult SetDo(int port, int value, int id = 1);
```

模拟量 IO

GetAi / GetAo

```
double GetAi(int port, int id = 1);
double GetAo(int port, int id = 1);
```

SetAo

```
CommandResult SetAo(int port, double value, int id = 1);
```

寄存器

GetRegisterValue / GetRegisterValues

```
double GetRegisterValue(int address, int id = 1);
std::vector<ClientRegisterInfo> GetRegisterValues(const std::vector<int>& addresses, int id = 1);
```

SetRegisterValue

```
CommandResult SetRegisterValue(int address, double value, int id = 1);
```

完整示例

```
#include "Codroid/client.hpp"

Codroid::CodroidClient robot;
robot.ConnectRemoteAndSwitchOn("192.168.8.136", 9001, "192.168.8.150");
```

```
// IO 操作
int di0 = robot.GetDi(0);
robot.SetDo(10, di0);

double ai0 = robot.GetAi(0);
robot.SetAo(0, 3.3);

// 寄存器操作
double value = robot.GetRegisterValue(49100);
robot.SetRegisterValue(49100, value + 1);

// 批量读取
std::vector<int> addresses = {49100, 49101, 49102};
auto values = robot.GetRegisterValues(addresses);

robot.Disconnect();
```

辅助工具 API 参考

主题订阅

SubscribePublishTopic

```
ClientPublishSubscription SubscribePublishTopic(
    std::string topicTy,
    std::function<void(const ClientPublishNotification&)> handler,
    int tc_milliseconds = 100);
```

可用主题

主题	说明
publish/ProjectState	工程运行状态
publish/VarUpdate	变量更新
publish/RobotStatus	机器人状态
publish/RobotPosture	机器人位姿
publish/RobotCoordinate	坐标系
publish/Log	日志
publish/Error	错误

PublishTopics 常量

```
struct PublishTopics {
    static constexpr const char* ProjectState = "publish/ProjectState";
    static constexpr const char* VarUpdate = "publish/VarUpdate";
    static constexpr const char* RobotStatus = "publish/RobotStatus";
    static constexpr const char* RobotPosture = "publish/RobotPosture";
    static constexpr const char* RobotCoordinate = "publish/RobotCoordinate";
    static constexpr const char* Log = "publish/Log";
    static constexpr const char* Error = "publish/Error";
};
```


全局变量

GetGlobalVars / SaveGlobalVars / RemoveGlobalVars

```
nlohmann::json GetGlobalVars(int id = 1);
CommandResult SaveGlobalVars(const std::map<std::string, Variable>& vars, int id = 1);
CommandResult RemoveGlobalVars(const std::vector<std::string>& names, int id = 1);
```

运动学

ForwardKinematics / InverseKinematics

```
std::vector<double> ForwardKinematics(const FKParams& params, int id = 1);
std::vector<double> InverseKinematics(const IKParams& params, int id = 1);
```

CalculateRelativePose

```
std::vector<double> CalculateRelativePose(const RelativePoseParams& params, int id = 1);
```

CposToCpos / CposToCposPose (v2.1.7+)

坐标系转换：将 TCP 位姿从坐标系1+工具1 转换到坐标系2+工具2。协议 `Robot/cpostocpos`。

```
// 返回原始 double 数组
std::vector<double> CposToCpos(const std::vector<double>& cp,
                               const std::vector<double>& coor1, const std::vector<double>&
tool1,
                               const std::vector<double>& coor2, const std::vector<double>&
tool2,
                               int id = 1);

// 返回 CartesianPoint
CartesianPoint CposToCposPose(const CartesianPoint& cp,
                              const std::vector<double>& coor1, const std::vector<double>&
tool1,
                              const std::vector<double>& coor2, const std::vector<double>&
tool2,
                              int id = 1);
```

参数	说明
cp	当前 TCP 位姿 [x, y, z, a, b, c] (mm+度)
coor1	源坐标系 [x, y, z, a, b, c]
tool1	源工具 [x, y, z, a, b, c]
coor2	目标坐标系 [x, y, z, a, b, c]

参数	说明
<code>tool2</code>	目标工具 <code>[x,y,z,a,b,c]</code>

```
auto result = robot.CposToCpos({400,200,500,180,0,90},
                                {0,0,0,0,0,0}, {0,0,0,0,0,0},
                                {100,0,0,0,0,0}, {0,0,100,0,0,0});
```

RS485 通信 (v2.1.7+)

Rs485Init

```
CommandResult Rs485Init(int baudrate, RS485StopBits stopBit = RS485StopBits::One,
                        RS485Parity parity = RS485Parity::None, int id = 1);
```

初始化末端 RS485 通信。

Rs485Flush

```
CommandResult Rs485Flush(int id = 1);
```

清空 RS485 读取缓冲区。

Rs485Read

```
nlohmann::json Rs485Read(int length, int timeout = 5000, int id = 1);
```

读取 RS485 数据。 `length` 最大 128 字节， `timeout` 最大 3000ms。

Rs485Write

```
CommandResult Rs485Write(const std::vector<uint8_t>& data, int id = 1);
```

写入 RS485 数据。 `data` 最大 127 字节。

```
robot.Rs485Init(115200, Codroid::RS485StopBits::One, Codroid::RS485Parity::None);
robot.Rs485Flush();
auto data = robot.Rs485Read(7, 1000);
robot.Rs485Write({0x01, 0x03, 0x00, 0x00, 0x00, 0x01, 0x84, 0x0A});
```

工程控制扩展（v2.1.7+）

SetStartLine / ClearStartLine

```
CommandResult SetStartLine(int line, int id = 1);  
CommandResult ClearStartLine(int id = 1);
```

设置/清除工程执行启动行。

GetProjectVar

```
nlohmann::json GetProjectVar(int id = 1);
```

获取当前工程变量值（仅在工程运行中有效）。

GetGlobalVarsCatalog

```
nlohmann::json GetGlobalVarsCatalog(int id = 1);
```

获取全局变量目录（与 `GetGlobalVars` 同协议，结构化解析为 `变量名 → {Value, Remark}`）。

寄存器扩展（v2.1.7+）

SetExtendArrayType / RemoveExtendArray

```
CommandResult SetExtendArrayType(int index, ExtendArrayType type, int id = 1);  
CommandResult RemoveExtendArray(int index, int id = 1);
```

设置/重置扩展数组元素类型。

机器人设置扩展（v2.1.7+）

SaveUserCoordinateFrames

```
CommandResult SaveUserCoordinateFrames(const std::vector<ClientRobotFrame>& frames, int id = 1);
```

直接下发完整用户坐标系表（19.6），与 `SaveToolFrames` / `SavePayloadFrames` 对齐。

控制台 UTF-8

ConsoleUtf8

```
#include "Codroid/console_utf8.hpp"

// Windows 控制台 UTF-8 支持
#ifdef _WIN32
Codroid::ConsoleUtf8::InitConsoleUtf8();
#endif
```

完整示例

```
#include "Codroid/client.hpp"

// 主题订阅
auto sub = robot.SubscribePublishTopic(
    Codroid::PublishTopics::RobotStatus,
    [](const Codroid::ClientPublishNotification& n) {
        std::cout << "状态: " << n.db_json << std::endl;
    }
);

// 全局变量
robot.SaveGlobalVars({
    {"counter", Codroid::Variable(42, "计数器")},
    {"name", Codroid::Variable(std::string("test"), "名称")}
});

// 运动学
Codroid::FKParams fk({0, 0, 90, 0, 90, 0});
auto tcp = robot.ForwardKinematics(fk);

Codroid::IKParams ik({400, 0, 300, 180, 0, 0});
ik.rj = {0, 0, 90, 0, 90, 0};
auto joints = robot.InverseKinematics(ik);
```